

**LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 1**



ANDROID BASIC WITH KOTLIN

Oleh:

ERISCHA MARSELA

NIM. 2410817120022

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
APRIL 2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE
MODUL 1

Laporan Praktikum Pemrograman Mobile Modul 1: Android Basic With Kotlin Sederhana ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman II. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : ERISCHA MARSELA

NIM : 2410817120022

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Erza Raihan
NIM. 2310817210027

Irham Maulani Abdul Gani, S.Kom.,
M.Kom.
NIP. 199710312025061009

DAFTAR ISI

LEMBAR PENGESAHAN	2
DAFTAR ISI.....	3
DAFTAR TABEL.....	4
DAFTAR GAMBAR.....	5
SOAL 1	6
A. SOURCE CODE.....	9
B. OUTPUT CODE.....	18
C. PEMBAHASAN	19
TAUTAN GIT	21
ADDITIONAL.....	21

DAFTAR TABEL

Tabel 1. Source Code MainActivity.kt XML Soal 1	9
Tabel 2. Source Code Activity_main.xml Soal 1	11
Tabel 3. Source Code MainActivity.kt Compose Soal 1	13

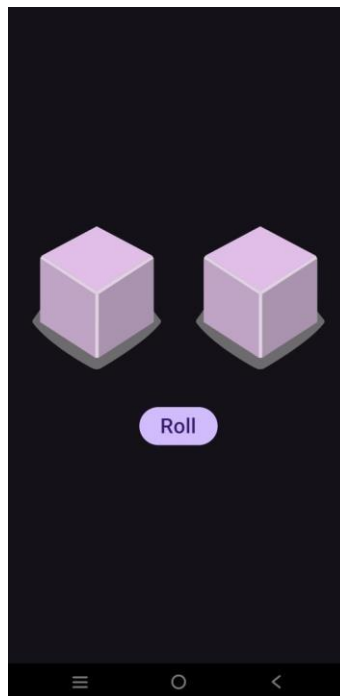
DAFTAR GAMBAR

Gambar 1. Tampilan Awal Aplikasi	6
Gambar 2. Tampilan Dadu Setelah Di-Roll.....	7
Gambar 3. Tampilan Roll Dadu Double	8
Gambar 4. Screenshot Hasil Jawaban Soal 1 Compose.....	18
Gambar 5. Screenshot Hasil Jawaban Soal 1 Compose.....	18
Gambar 6. Screenshot Hasil Jawaban Soal 1 XML	19

SOAL 1

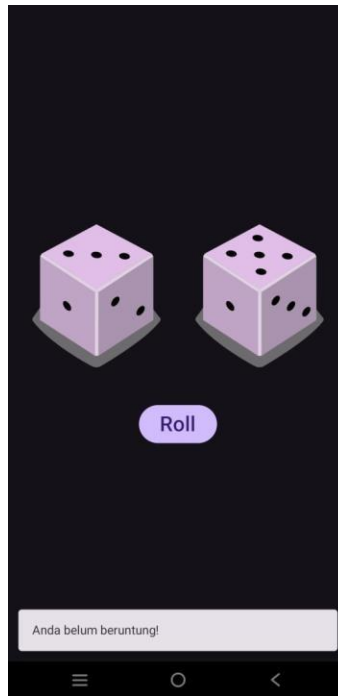
Buatlah sebuah aplikasi yang dapat menampilkan 2 buah dadu yang dapat berubah-ubah tampilannya pada saat user menekan tombol “Roll”. Aturan aplikasi yang akan dibangun adalah sebagaimana berikut:

1. Tampilan awal aplikasi setelah dijalankan akan menampilkan 2 buah dadu kosong seperti dapat dilihat pada Gambar 1.



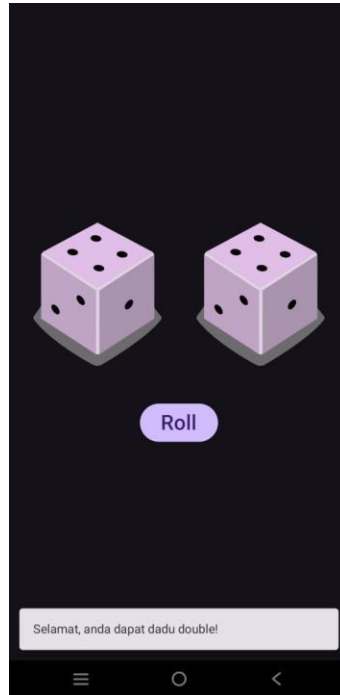
Gambar 1. Tampilan Awal Aplikasi

2. Setelah user menekan tombol “Roll” maka masing-masing dadu akan memperlihatkan sisi dadunya dengan angka antara 1 s/d 6. Apabila user mendapatkan nilai dadu yang berbeda antara Dadu 1 dengan Dadu 2, maka aplikasi akan menampilkan pesan “Anda belum beruntung!” seperti yang dapat dilihat pada Gambar 2.



Gambar 2. Tampilan Dadu Setelah Di-Roll

3. Apabila user mendapatkan nilai dadu yang sama antara Dadu 1 dan Dadu 2 atau nilai double, maka aplikasi akan menampilkan pesan “Selamat, anda dapat dadu double!” seperti yang dapat dilihat pada Gambar 3.



Gambar 3. Tampilan Roll Dadu Double

4. Buatlah aplikasi tersebut menggunakan XML dan Jetpack Compose.
5. Upload aplikasi yang telah anda buat ke dalam repository GitHub ke dalam **folder Modul 1 dalam bentuk Project**. Jangan lupa untuk melakukan **Clean Project** sebelum mengupload pekerjaan anda pada repository.
6. Untuk gambar dadu dapat didownload pada link berikut:
https://drive.google.com/file/d/14V3qXGdFnuoYN4AGd_9SgFh8kw8X9ySm/view?usp=sharing

Additional:

ESSAY

1. Riset dan *cite* **artikel/jurnal/buku** mengenai Imperative Programming dan Declarative Programming beserta penerapannya di pembuatan UI. Buatlah opini paradigma apa yang kalian prefer berdasarkan riset yang sudah dilakukan.
**
2. Apa yang membedakan OOP pada kotlin dengan OOP pada java? *
3. Sebutkan, dan jelaskan mengenai SOLID Principles *

4. Apa perbedaan antara Class dan Data Class pada Kotlin ?

**** = 2 sumber**

*** = 1 sumber**

A. SOURCE CODE

MainActivity.Kt XML

Tabel 1. Source Code MainActivity.kt XML Soal 1

1.	package com.example.module1xml
2.	
3.	import android.os.Bundle
4.	import android.widget.Button
5.	import android.widget.ImageView
6.	import android.widget.Toast
7.	import androidx.appcompat.app.AppCompatActivity
8.	
9.	class MainActivity : AppCompatActivity() {
10.	private lateinit var leftDiceView: ImageView
11.	private lateinit var rightDiceView: ImageView
12.	private lateinit var btnActionRoll: Button
13.	
14.	override fun onCreate(savedInstanceState: Bundle?) {
15.	super.onCreate(savedInstanceState)
16.	setContentView(R.layout.activity_main)
17.	
18.	initView()
19.	setupListener()
20.	}
21.	

22.	private fun initView() {	
23.	leftDiceView	=
	findViewById(R.id.iv_left_dice)	
24.	rightDiceView	=
	findViewById(R.id.iv_right_dice)	
25.	btnActionRoll	=
	findViewById(R.id.btn_action_roll)	
26.	}	
27.		
28.	private fun setupListener() {	
29.	btnActionRoll.setOnClickListener {	
30.	executeDiceRoll()	
31.	}	
32.	}	
33.		
34.	private fun executeDiceRoll() {	
35.	val firstValue = (1..6).random()	
36.	val secondValue = (1..6).random()	
37.		
38.	leftDiceView.setImageResource(resolveDiceDrawable	
	(firstValue))	
39.	rightDiceView.setImageResource(resolveDiceDrawabl	
	e(secondValue))	
40.		
41.	val resultMessage = if (firstValue ==	
	secondValue) {	
42.	getString(R.string.double_message)	
43.	} else {	
44.	getString(R.string.bad_message)	
45.	}	
46.		

47.	Toast.makeText(this, resultMessage,
48.	Toast.LENGTH_SHORT).show()
49.	}
50.	private fun resolveDiceDrawable(value: Int):
51.	Int {
52.	return when (value) {
53.	1 -> R.drawable.dice_1
54.	2 -> R.drawable.dice_2
55.	3 -> R.drawable.dice_3
56.	4 -> R.drawable.dice_4
57.	5 -> R.drawable.dice_5
58.	else -> R.drawable.dice_6
59.	}
60.	}

Activity_main.xml

Tabel 2. Source Code Activity_main.xml Soal 1

1.	<?xml version="1.0" encoding="utf-8"?>
2.	<androidx.constraintlayout.widget.ConstraintLay
3.	out
4.	xmlns:android="http://schemas.android.com/apk/r
5.	es/android"
6.	xmlns:app="http://schemas.android.com/apk/res-
7.	auto"
8.	android:layout_width="match_parent"
	android:layout_height="match_parent"
	android:padding="16dp">

9.	<ImageView
10.	android:id="@+id/iv_left_dice"
11.	android:layout_width="120dp"
12.	android:layout_height="120dp"
13.	android:layout_marginEnd="8dp"
14.	android:src="@drawable/dice_1"
15.	app:layout_constraintBottom_toTopOf="@+id/btn_action_roll"
16.	app:layout_constraintEnd_toStartOf="@+id/iv_right_dice"
17.	app:layout_constraintHorizontal_chainStyle="packed"
18.	app:layout_constraintStart_toStartOf="parent"
19.	app:layout_constraintTop_toTopOf="parent"
20.	app:layout_constraintVertical_chainStyle="packed" />
21.	
22.	<ImageView
23.	android:id="@+id/iv_right_dice"
24.	android:layout_width="120dp"
25.	android:layout_height="120dp"
26.	android:layout_marginStart="8dp"
27.	android:src="@drawable/dice_1"
28.	app:layout_constraintBottom_toBottomOf="@+id/iv_left_dice"
29.	app:layout_constraintEnd_toEndOf="parent"

30.	app:layout_constraintStart_toEndOf="@+id/iv_left_dice"
31.	app:layout_constraintTop_toTopOf="@+id/iv_left_dice" />
32.	
33.	<Button
34.	android:id="@+id/btn_action_roll"
35.	android:layout_width="wrap_content"
36.	android:layout_height="wrap_content"
37.	android:layout_marginTop="32dp"
38.	android:text="@string/roll"
39.	app:layout_constraintBottom_toBottomOf="parent"
40.	app:layout_constraintEnd_toEndOf="parent"
41.	app:layout_constraintStart_toStartOf="parent"
42.	app:layout_constraintTop_toBottomOf="@+id/iv_left_dice" />
43.	
44.	</androidx.constraintlayout.widget.ConstraintLayout>

MainActivity.Kt Compose

Tabel 3. Source Code MainActivity.kt Compose Soal 1

1.	package com.example.module1
2.	
3.	import android.os.Bundle
4.	import androidx.activity.ComponentActivity
5.	import androidx.activity.compose.setContent
6.	import androidx.compose.foundation.Image

7.	import androidx.compose.foundation.layout.Arrangement
8.	import androidx.compose.foundation.layout.Column
9.	import androidx.compose.foundation.layout.Row
10.	import androidx.compose.foundation.layout.Spacer
11.	import androidx.compose.foundation.layout.fillMaxSize
12.	import androidx.compose.foundation.layout.height
13.	import androidx.compose.foundation.layout.width
14.	import androidx.compose.foundation.layout.wrapContentSize
15.	import androidx.compose.material3.Button
16.	import androidx.compose.material3.Text
17.	import androidx.compose.runtime.Composable
18.	import androidx.compose.runtime.getValue
19.	import androidx.compose.runtime.mutableStateOf
20.	import androidx.compose.runtime.remember
21.	import androidx.compose.runtime.setValue
22.	import androidx.compose.ui.Alignment
23.	import androidx.compose.ui.Modifier
24.	import androidx.compose.ui.res.painterResource
25.	import androidx.compose.ui.res.stringResource
26.	import androidx.compose.ui.tooling.preview.Preview
27.	import androidx.compose.ui.unit.dp
28.	
29.	class MainActivity : ComponentActivity() {
30.	override fun onCreate(savedInstanceState: Bundle?) {
31.	super.onCreate(savedInstanceState)
32.	setContent {
33.	TwoDiceRollerScreen()
34.	}

```

35.     }
36. }
37.
38. fun getDiceDrawable(diceValue: Int): Int {
39.     return when (diceValue) {
40.         1 -> R.drawable.dice_1
41.         2 -> R.drawable.dice_2
42.         3 -> R.drawable.dice_3
43.         4 -> R.drawable.dice_4
44.         5 -> R.drawable.dice_5
45.         else -> R.drawable.dice_6
46.     }
47. }
48.
49. @Preview(showBackground = true)
50. @Composable
51. fun DiceAppPreview() {
52.     TwoDiceRollerScreen()
53. }
54.
55. @Composable
56. fun TwoDiceRollerScreen(
57.     modifier: Modifier = Modifier
58.         .fillMaxSize()
59.         .wrapContentSize(Alignment.Center)
60. ) {
61.     var firstDiceValue by remember {
62.         mutableStateOf(1) }
63.     var secondDiceValue by remember {
64.         mutableStateOf(1) }
65.     var statusText by remember { mutableStateOf("") }

```

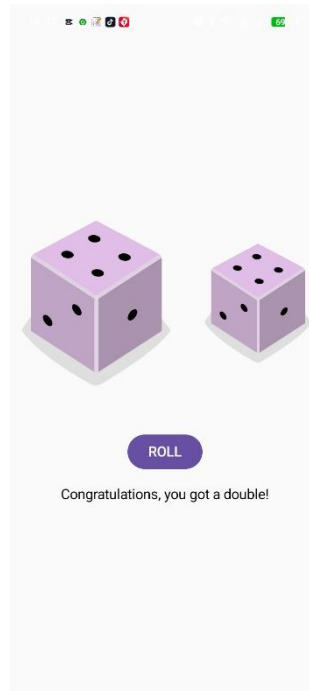
64.		
65.	val leftDiceImage	=
	getDiceDrawable (firstDiceValue)	
66.	val rightDiceImage	=
	getDiceDrawable (secondDiceValue)	
67.		
68.	val winMessage	=
	stringResource (R.string.double_message)	
69.	val loseMessage	=
	stringResource (R.string.bad_message)	
70.		
71.	Column (	
72.	modifier = modifier,	
73.	horizontalAlignment	=
	Alignment.CenterHorizontally	
74.	) {	
75.	Row (	
76.	verticalAlignment	=
	Alignment.CenterVertically,	
77.	horizontalArrangement	=
	Arrangement.Center	
78.	) {	
79.	Image (	
80.	painter	=
	painterResource (leftDiceImage),	
81.	contentDescription = "Left Dice:	
	\$firstDiceValue"	
82.	)	
83.		
84.	Spacer (modifier	=
	Modifier.width (16.dp))	
85.		
86.	Image (	
87.	painter	=
	painterResource (rightDiceImage),	


```

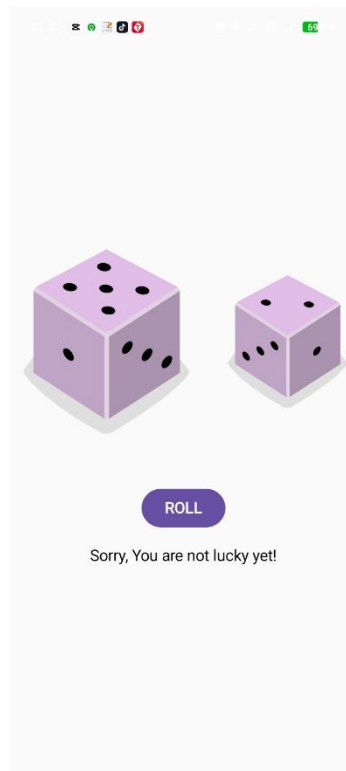
88.         contentDescription = "Right Dice:
$secondDiceValue"
89.     )
90. }
91.
92.     Spacer(modifier = Modifier.height(24.dp))
93.
94.     Button(
95.         onClick = {
96.             firstDiceValue = (1..6).random()
97.             secondDiceValue = (1..6).random()
98.
99.             statusText = if (firstDiceValue ==
secondDiceValue) {
100.                 winMessage
101.             } else {
102.                 loseMessage
103.             }
104.         }
105.     ) {
106.         Text(text =
stringResource(R.string.roll))
107.     }
108.
109.     Spacer(modifier = Modifier.height(16.dp))
110.
111.     Text(text = statusText)
112. }
113. }

```

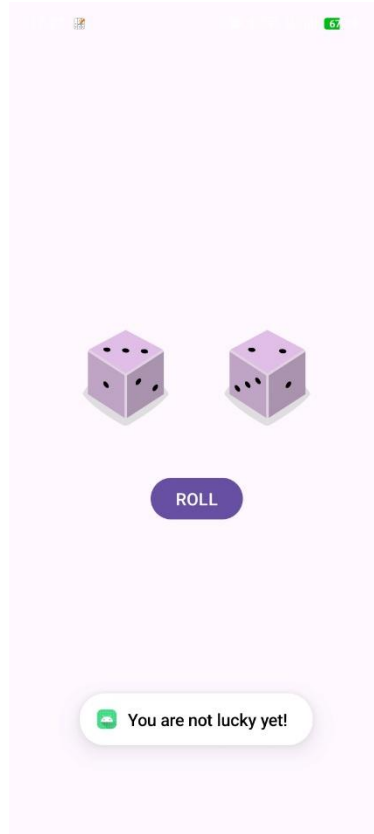
B. OUTPUT CODE



Gambar 4. Screenshot Hasil Jawaban Soal 1 Compose



Gambar 5. Screenshot Hasil Jawaban Soal 1 Compose



Gambar 6. Screenshot Hasil Jawaban Soal 1 XML

C. PEMBAHASAN

MainActivity.ky XML

Pada MainActivity.kt versi XML, kita membangun logika sederhana yang menampilkan dua gambar dadu beserta sebuah tombol untuk melakukan roll (mengacak) angka dadu. Aplikasi ini berjalan dengan menggunakan AppCompatActivity. Secara alur kerja, saat aplikasi pertama kali dibuka (di dalam onCreate), kita memanggil method initView() untuk menghubungkan variabel dengan komponen UI di XML menggunakan findViewById(). Setelah itu, setupListener() dipanggil untuk mendengarkan aksi dari user ketika tombol btn_action_roll diklik.

Ketika tombol tersebut ditekan, fungsi executeDiceRoll() akan dieksekusi. Nah, di sinilah logika utamanya berjalan yaitu, kita menggunakan fungsi (1..6).random() untuk menghasilkan angka acak antara 1 sampai 6 untuk masing-masing dadu. Angka acak tersebut kemudian dilempar ke dalam fungsi resolveDiceDrawable() yang berisi blok when. Fungsinya adalah untuk

menentukan gambar drawable mana yang harus dipasang ke ImageView sesuai dengan angkanya. Terakhir, kita menggunakan kondisional if-else untuk mengecek apakah nilai dadu pertama sama dengan dadu kedua. Jika nilainya kembar (double), sistem akan mengambil pesan dari double_message. Jika beda, pesan yang diambil adalah bad_message. Pesan tersebut kemudian ditampilkan ke user melalui Toast.

Activity_main.xml

Pemilihan ConstraintLayout ini sangat membantu karena memungkinkan kita menata elemen-elemen UI secara lebih fleksibel dengan menetapkan posisi relatif antar elemen, sehingga UI bisa lebih efisien dan adaptif di berbagai ukuran layar. Di dalamnya terdapat tiga komponen utama yaitu, dua buah ImageView (iv_left_dice dan iv_right_dice) yang digunakan untuk menampilkan gambar dadu dengan ukuran statis 120dp. Agar posisinya rapi berjejeran di tengah layar, kita memanfaatkan properti layout_constraintHorizontal_chainStyle="packed". Serta, satu buah Button (btn_action_roll): Tombol pelempar dadu ini diposisikan di bawah chain kedua dadu tadi dengan margin top sebesar 32dp agar tidak terlalu menempel dengan gambar dadu di atasnya.

MainActivity.kt Compose

Class utama kita mewarisi ComponentActivity, yang dimana fungsi setContent langsung memanggil UI utama kita yaitu composable function TwoDiceRollerScreen(). Dalam merakit UI, kita menggunakan Column agar tata letaknya berjejer ke bawah (vertikal), dan menempatkan kedua gambar dadu di dalam Row agar berbaris ke samping (horizontal). Kita juga memakai Spacer untuk memberikan jarak (padding) antar komponen.

Hal yang paling penting di Compose adalah manajemen state. Nilai dadu pertama, dadu kedua, dan teks pesan dibungkus di dalam remember dan mutableStateOf. Hal ini memastikan bahwa UI akan recompose atau merespons secara otomatis ketika nilainya berubah saat tombol ditekan. Pada event onClick di komponen Button, dadu kembali diacak menggunakan (1..6).random(). Sama seperti di versi XML, hasil pengecekan apakah dadunya double atau tidak akan menentukan pesan (stringResource) yang diupdate ke state statusText. Penggunaan

stringResource() ini sangat baik karena membuat aplikasi lebih fleksibel jika ke depannya ingin ditambahkan dukungan ragam bahasa (internationalization).

TAUTAN GIT

<https://github.com/ERISCHA24/PRAKTIKUMMOBILE.git>

ADDITIONAL

1. Riset dan *cite* artikel/jurnal/buku mengenai Imperative Programming dan Declarative Programming beserta penerapannya di pembuatan UI. Buatlah opini paradigma apa yang kalian prefer berdasarkan riset yang sudah dilakukan. **

Pemrograman imperatif dan deklaratif memiliki pendekatan logika yang sangat berbeda, terutama saat kita mengimplementasikannya pada pembuatan antarmuka pengguna (UI). Pemrograman imperatif mengandalkan pendekatan step-by-step, di mana kita sebagai developer harus memberikan instruksi spesifik mengenai bagaimana UI tersebut harus di-render sekaligus mengontrol state-nya secara manual. Sebaliknya, pemrograman deklaratif jauh lebih simpel karena berfokus pada apa yang ingin kita tampilkan; kita cukup mendeskripsikan kondisi akhir atau state dari antarmuka tersebut tanpa perlu menyusun urutan operasinya.

Opini saya lebih prefer menggunakan paradigma deklaratif untuk development UI. Alasannya karena UI deklaratif secara signifikan meningkatkan kemudahan maintenance dan testing pada kode kita. Pendekatan ini juga sangat mendukung separation of concerns dengan memisahkan logika antarmuka dari detail implementasi di belakangnya. Terlebih lagi, rata-rata framework modern untuk mobile development saat ini memang sudah difokuskan ke arah deklaratif.

Sumber =

* **Smith, J., Guo, X., & Bansal, A. (2022).** *A Predicate Construct for Declarative Programming in Imperative Languages.* ([A Predicate Construct for Declarative Programming in Imperative Languages](#))

* **Zhou, T., Zhao, Y., Hou, X., Sun, X., Chen, K., & Wang, H. (2024).** *Bridging Design and Development with Automated Declarative UI Code Generation.* ([Bridging Design and Development with Automated Declarative UI Code Generation](#))

2. Apa yang membedakan OOP pada kotlin dengan OOP pada java? *

Jika membandingkan konsep Object-Oriented Programming (OOP) antara Kotlin dan Java, perbedaan yang paling terasa ada pada penulisan sintaks dan jumlah boilerplate code-nya. Java menggunakan pendekatan OOP klasik yang cenderung sangat panjang, sehingga kita diharuskan melakukan deklarasi secara ketat. Walaupun aman, hal ini sering kali membuat basis kode kita menjadi sangat besar. Berbeda halnya dengan Kotlin, yang sejak awal didesain dengan sintaks yang jauh lebih ringkas dan ekspresif. Selain itu, Kotlin tidak hanya membatasi kita pada OOP, tetapi juga menggabungkannya dengan paradigma pemrograman fungsional, sehingga arsitektur yang kita bangun bisa lebih fleksibel.

Sumber =

* **Petriv, Nicolas (2024).** *A Comparative Study of Java and Kotlin for Android Mobile Application Development.* ([COMPARATIVE-STUDY-OF-JAVA-AND-KOTLIN-FOR-ANDROID-DEVELOPMENT/Comparative study of Java and Kotlin for Android development \(scientific article\).pdf at main · NicolasPetrov/COMPARATIVE-STUDY-OF-JAVA-AND-KOTLIN-FOR-ANDROID-DEVELOPMENT](#))

3. Sebutkan, dan jelaskan mengenai SOLID Principles *

Prinsip SOLID ibarat guideline wajib yang harus kita pegang agar kode yang kita tulis menjadi lebih rapi, scalable, dan mudah dikelola ke depannya.

Prinsip ini mencakup lima aturan utama. Pertama ada Single Responsibility Principle (SRP), yang menegaskan bahwa sebuah kelas hanya boleh memiliki satu tanggung jawab atau satu alasan saja untuk berubah. Kedua, Open/Closed Principle (OCP), yang berarti entitas program kita harus selalu terbuka kalau mau ditambahin fitur baru, tapi tertutup dari modifikasi kode intinya. Ketiga, Liskov Substitution Principle (LSP), di mana objek dari sebuah subclass harus bisa menggantikan superclass-nya tanpa mengacaukan fungsionalitas sistem. Keempat, Interface Segregation Principle (ISP), yang mengingatkan kita untuk memecah interface yang terlalu besar menjadi bagian-bagian yang lebih kecil dan spesifik, agar klien tidak perlu bergantung pada method yang sebenarnya tidak mereka pakai. Terakhir adalah Dependency Inversion Principle (DIP), yang intinya menekankan bahwa modul tingkat tinggi tidak boleh bergantung pada modul tingkat rendah tapi keduanya harus sama-sama bergantung pada abstraksi.

Sumber =

* **Martin, Robert C. (2002).** *Agile Software Development: Principles, Patterns, and Practices*. ([Software_book/Agile.Principles.Patterns.and.Practices.pdf at main · rahff/Software_book](#))

4. Apa perbedaan antara Class dan Data Class pada Kotlin ?

Class biasa adalah fondasi standar dari OOP di mana kita punya kendali penuh tapi harus menuliskan semuanya secara manual, mulai dari properti, metode, sampai membuat fungsi-fungsi getter dan setter. Nah, Kotlin menyediakan Data Class, yaitu tipe kelas spesial yang memang dirancang khusus hanya untuk merepresentasikan dan menyimpan objek data sederhana. Keuntungan utama memakai Data Class adalah Kotlin bakal otomatis nge-generate fungsi-fungsi standar atau kode boilerplate di belakang layar. Efeknya, kode kita tidak hanya menjadi lebih cepat untuk ditulis.